

Optimization of IMU-Based Tele-Imitation Systems for Industrial Robotics: Architectural Analysis and Latency Compensation

1. Introduction and Operational Context

The paradigm of Programming by Demonstration (PbD), specifically within the sub-discipline of Tele-Imitation, represents a fundamental shift in industrial robotics from explicit code-based instruction to intuitive, motion-based teaching. In this operational model, the kinetic intent of a human operator is captured via a handheld interface and mapped in real-time to the kinematic chain of an industrial manipulator, such as a COMAU robot. This report presents an exhaustive technical analysis of a proposed wireless tele-imitation system utilizing the Espressif ESP32 microcontroller and the Bosch BNO055 Inertial Measurement Unit (IMU), communicating via the Robot Operating System 2 (ROS 2) middleware.

The critical engineering challenge identified for this system is the requirement for high-frequency orientation tracking—specifically a target floor of 50 Hz—coupled with low-latency transmission to ensure stability in the human-robot control loop. Unlike open-loop control systems where latency results merely in a delayed execution, tele-imitation constitutes a closed-loop system where the operator visualizes the robot's response (via RViz2 or direct line-of-sight) and adjusts their hand motion accordingly. Latencies exceeding the perceptual threshold of approximately 100 to 150 milliseconds can induce Pilot-Induced Oscillations (PIO), where the operator's corrective movements phase-lag behind the robot's actual position, leading to instability, hazardous overshoots, and degraded task performance.

To achieve the requisite performance on resource-constrained embedded hardware, a naive implementation of "reading a sensor and sending a packet" is insufficient. The architecture requires a holistic optimization strategy that permeates every layer of the Open Systems Interconnection (OSI) model: from the physical electrical characteristics of the I2C bus and the silicon-level quirks of the sensor's internal fusion engine, through the real-time scheduling of the operating system (RTOS), up to the transport layer Quality of Service (QoS) policies in the Data Distribution Service (DDS), and finally into the predictive signal processing on the host computer. This document details the specific engineering interventions required at each of these stages to guarantee a deterministic 50 Hz throughput and minimize end-to-end latency.

2. Sensor Hardware Dynamics and Interface

Optimization

The selection of the Bosch BNO055 imposes specific constraints that define the behavior of the entire embedded subsystem. Unlike "dumb" sensors that simply output raw voltage values from a micro-electromechanical system (MEMS) structure, the BNO055 is a System-in-Package (SiP) containing a triaxial accelerometer, gyroscope, and magnetometer, along with a dedicated 32-bit microcontroller running the BSX3.0 FusionLib software. While this architecture offloads complex Kalman filtering from the host ESP32, it introduces significant non-determinism in data availability and bus timing.

2.1 The Physics of I2C Clock Stretching

The primary bottleneck in the interface between the ESP32 and the BNO055 is the phenomenon of I2C clock stretching. The BNO055's internal processor prioritizes sensor fusion calculations over external communication. When the ESP32 master initiates a read transaction to retrieve the quaternion orientation data (typically registers 0x20 through 0x27), the BNO055 may not have the data immediately available in its output buffer, or it may be in a critical section of its fusion loop updating the state vector. To prevent the master from reading invalid or torn data, the BNO055 holds the Serial Clock (SCL) line low (voltage $< V_{IL}$) after the master releases it.

This mechanism effectively pauses the I2C transaction at the bit level. Empirical analysis and community documentation indicate that the BNO055 is an aggressive user of clock stretching, with stretch durations frequently exceeding 500 microseconds and occasionally reaching into the millisecond range depending on the selected operating mode (e.g., NDOF vs. IMU mode).¹ In standard microcontroller environments, I2C drivers often employ a hardware or software watchdog timer to detect bus lockups. If the slave device holds the clock low for a duration exceeding this timeout, the master assumes a bus error and aborts the transaction. In the context of the ESP32, this interaction is complicated by specific hardware implementations of the I2C peripheral.

2.2 ESP32 Silicon Characteristics and Timeout Configuration

The implementation of the I2C timeout mechanism in the ESP32 architecture varies significantly between chip revisions and ESP-IDF (IoT Development Framework) versions. A distinct "silicon quirk" exists in the standard ESP32 (Xtensa LX6) compared to the newer ESP32-S3 variants, specifically regarding how the timeout register values are interpreted.

In the legacy ESP32 architecture, the I2C timeout register represents a linear count of APB (Advanced Peripheral Bus) clock cycles. The APB clock typically runs at 80 MHz. Therefore, a register value of N corresponds to a timeout duration $T_{out} = N \times (1 / 80,000,000)$. To accommodate a clock stretch of 10 milliseconds—a conservative safety margin for the BNO055—the timeout register must be set to approximately 800,000 cycles. However, standard driver implementations often default to much lower values, intended to

catch genuine bus errors rather than accommodate slow intelligent sensors. This mismatch results in ESP_ERR_TIMEOUT errors, dropped sensor frames, and a functional sample rate well below the 50 Hz target.³

Furthermore, newer variants like the ESP32-S3 utilize an exponential timeout scaling, where the register value N corresponds to a duration proportional to 2^N clock cycles. This discrepancy forces developers to be extremely precise in their driver configuration code. For the tele-imitation system using a standard ESP32, the optimization strategy mandates maximizing the timeout value to 0xFFFF (approximately 13ms at 80MHz) to prevent the driver from prematurely severing connection with the BNO055 during a fusion update cycle.

2.3 Electrical Signal Integrity and Bus Capacitance

Beyond the protocol layer, the physical layer of the I2C bus presents challenges to high-frequency polling. The I2C bus relies on open-drain topology, where lines are pulled high by resistors and pulled low by active transistors. The rise time of the signal (t_r) is determined by the RC time constant of the bus: $\tau = R_{\text{pullup}} \times C_{\text{bus}}$.

The ESP32 features internal pull-up resistors, but their resistance is relatively high (typically varying between 30 k Ω and 50 k Ω). When combined with the parasitic capacitance of wires, breadboards, or long traces in a handheld device, the resulting time constant can be significant. This leads to "shark-fin" waveforms rather than sharp square waves. The BNO055, upon releasing the clock stretch, relies on the SCL line returning to a high logic level (V_{IH}) quickly to resume the transaction. If the rise time is too slow, the BNO055 or the ESP32 may misinterpret the timing, leading to arbitration losses or data corruption.⁴

To mitigate this, the system design must explicitly disable the ESP32's internal pull-ups and utilize stronger external pull-up resistors. A resistance value between 2.2 k Ω and 4.7 k Ω is recommended. This lowers the RC time constant, sharpening the signal edges and ensuring robust communication even when the BNO055 is operating at the limits of its timing specifications. The following table summarizes the electrical configuration parameters required for stability.

Parameter	Standard Configuration	Optimized Configuration	Rationale
I2C Clock Speed	400 kHz	100 kHz (or lower)	Lower speeds reduce capacitive effects and give the BNO055 more processing margin

			per bit.
Pull-Up Resistors	Internal (~45 k Ω)	External (2.2 k Ω)	Sharpens signal rise times; critical for recovery from clock stretching.
Timeout Value	Default (~1000 cycles)	Max (0xFFFF)	Prevents driver resets during valid sensor fusion pauses.
Driver Implementation	Arduino Wire.h	Native ESP-IDF I2C	Granular control over timeouts, interrupt allocation, and buffer management.

2.4 Alternative Sensor Architectures

While the BNO055 is the specified sensor, it is worth noting that for future iterations or extreme latency requirements, the BNO085 represents a superior hardware choice. The BNO085 utilizes the same MEMS elements but features an upgraded coprocessor and supports the SHTP (Sensor Hub Transport Protocol) over SPI. SPI (Serial Peripheral Interface) is a push-pull protocol that eliminates the concept of clock stretching entirely, allowing for deterministic readout times typically under 1 millisecond.⁵ However, within the constraints of the BNO055, the I2C optimizations detailed above are mandatory to achieve the 50 Hz throughput requirement.

3. Firmware Architecture: Real-Time Multiprocessing

The firmware running on the ESP32 acts as the bridge between the physical sensing domain and the digital network domain. To achieve a stable 50 Hz loop rate, the firmware architecture must transition from a simple super-loop (common in hobbyist Arduino applications) to a deterministic, multi-threaded Real-Time Operating System (RTOS) design. The ESP32 is powered by FreeRTOS, which provides the necessary primitives for task scheduling, inter-process communication, and resource management.

3.1 Asymmetric Multiprocessing (AMP) on Dual Cores

The ESP32 integrates two Xtensa 32-bit LX6 microprocessors: the Protocol CPU (PRO_CPU or

Core 0) and the Application CPU (APP_CPU or Core 1). By default, the ESP-IDF framework pins the Wi-Fi stack and the LwIP (TCP/IP) stack to Core 0. These network tasks are highly interrupt-driven; receiving a Wi-Fi beacon or handling a TCP acknowledgment triggers high-priority interrupts that preempt running code.

If the sensor acquisition logic is placed in the same execution context as the Wi-Fi stack (e.g., the standard Arduino loop() which often runs on Core 1 but can be preempted, or worse, if forced onto Core 0), the blocking I2C transactions required by the BNO055 will be interrupted by network activity. This introduces "jitter"—variable latency between samples. For a tele-imitation system, jitter is detrimental because the host-side algorithms often assume a fixed time step (Δt) for integration and differentiation.

The optimized architecture employs **Asymmetric Multiprocessing (AMP)**.

- **Core 1 (Application Core):** This core is dedicated to the **Sensor Acquisition Task**. Its sole responsibility is to wake up at a precise interval (20ms), perform the blocking I2C read transaction, and place the data into a shared memory buffer. By isolating this task on Core 1, we ensure that I2C timing is not perturbed by the asynchronous nature of wireless networking.⁶
- **Core 0 (Protocol Core):** This core handles the **Network Transmission Task** (Micro-ROS Publisher) alongside the system Wi-Fi stack. This task retrieves data from the buffer and manages the serialization and UDP packet transmission. While this task may experience slight jitter due to Wi-Fi contention, the timestamped sensor data has already been captured deterministically on Core 1.

3.2 The Producer-Consumer Model

To decouple the synchronous sensor read from the asynchronous network transmission, a Producer-Consumer pattern is implemented using a FreeRTOS Queue.

The Queue Strategy:

A standard queue implementation might block the producer if the queue is full (backpressure). However, in a real-time tele-operation scenario, latency is more critical than completeness. We do not want to buffer 500ms of old hand movements if the network drops out; we want the robot to receive the current hand position immediately upon reconnection. Therefore, the standard FIFO (First-In-First-Out) behavior must be modified.

The Sensor Task (Producer) utilizes the xQueueOverwrite() API (available for queues of length 1) or a custom logic for ring buffers that overwrites the oldest data. This ensures that the queue always contains the single most recent quaternion estimate. The Network Task (Consumer) reads from this queue. If the network is congested and the Consumer is delayed, it will skip intermediate samples and jump straight to the freshest data when it recovers. This "lossy" behavior is preferable to "laggy" behavior in tele-operation.⁸

3.3 Interrupts vs. Polling for Synchronization

The BNO055 provides an interrupt pin (INT), but its behavior in fusion modes is often limited to specific events like "Any Motion" or "High G" rather than "New Data Ready".² Consequently, the system must rely on polling. To maintain a rigid 50 Hz schedule without drift, the Sensor Task must utilize the `vTaskDelayUntil()` function rather than `vTaskDelay()`.

`vTaskDelay(20)` pauses execution for 20 ticks *after* the function is called. If the I2C read takes 3ms, the total cycle time becomes 23ms, leading to a frequency drift (43 Hz instead of 50 Hz). `vTaskDelayUntil()` calculates the wake time based on the *previous* wake time, automatically subtracting the execution time of the I2C transaction. This guarantees that the start of each sampling period occurs exactly every 20ms, regardless of how long the BNO055 stretches the clock (provided it is less than 20ms).

3.4 Floating Point Optimization and FPU Usage

The orientation data from the BNO055 is essentially a vector of four scalar values (quaternion w, x, y, z). The ESP32's Xtensa LX6 core includes a hardware Floating Point Unit (FPU), but it is restricted to **single-precision (32-bit float)**. Double-precision (64-bit double) operations are emulated in software and are significantly slower—benchmarks suggest a performance penalty of 10x to 50x.¹¹

The Bosch sensor fusion output can be read as integers (LSB/unit) or doubles. The firmware must immediately convert these to float types. Developers must be vigilant with C literals; writing `1.0` implies a double, while `1.0f` implies a float. All math operations (normalization, coordinate frame rotation) performed on the ESP32 must strictly utilize single-precision functions (e.g., `sqrtof()` instead of `sqrt()`) to maintain the 50 Hz budget. Furthermore, compiler optimization flags (`-O2` or `-O3`) should be enabled in the `idf.py` menuconfig to allow the compiler to utilize the FPU pipeline efficiently.¹³

4. Middleware and Transport Layer: Micro-ROS

Micro-ROS serves as the bridge between the embedded domain of the ESP32 and the high-level control domain of ROS 2. It wraps the Micro XRCE-DDS (eXtremely Resource Constrained Environments Data Distribution Service) protocol, which is optimized for microcontroller constraints. The configuration of this middleware is the primary determinant of network-induced latency.

4.1 Transport Protocol Selection: UDP vs. Serial

While Micro-ROS supports transport over Serial (UART) and TCP, **UDP (User Datagram Protocol) over Wi-Fi** is the mandatory choice for this application.

- **Serial:** While low latency, it requires a physical tether (USB cable), defeating the "tele-imitation" goal of unencumbered hand motion.
- **TCP:** TCP provides reliable, ordered delivery. In the event of a lost packet, TCP stops the stream and requests retransmission. In a 50 Hz control loop, a packet that arrives 100ms

late due to retransmission is useless; the robot has already moved past that point in time. TCP introduces variable latency (jitter) that is difficult to filter.

- **UDP:** UDP provides "best-effort" delivery. It sends packets without waiting for acknowledgments. If a packet is lost, it is lost forever, and the system processes the next one. This constant-flow characteristic is ideal for real-time control.

4.2 Quality of Service (QoS) Policies

ROS 2 offers granular QoS settings that map down to the DDS layer. For tele-imitation, the default "Reliable" settings are counter-productive. The system must be configured for **Best Effort**.

QoS Policy	Reliable (Default)	Best Effort (Optimized)	Impact on Tele-Imitation
Reliability	Guarantees delivery via ACKs and retries.	No guarantees; fire-and-forget.	Best Effort eliminates blocking on the ESP32 while waiting for WiFi ACKs, reducing latency spikes.
History	KEEP_ALL or KEEP_LAST (Depth > 1).	KEEP_LAST (Depth = 1).	Storing history is unnecessary RAM usage; we only care about the instantaneous state.
Durability	TRANSIENT_LOCAL (Persists for late joiners).	VOLATILE (No persistence).	Volatile ensures that a reconnecting robot doesn't immediately process stale, buffered commands.

Implementation Detail:

The initialization of the publisher in the ESP32 firmware must explicitly use the Best Effort APIs. Using `rcl_publisher_init_default` typically configures Reliable QoS. The correct function call is `rcl_publisher_init_best_effort`. Furthermore, the subscriber on the ROS 2 host

computer must use a compatible QoS profile. In ROS 2, a Reliable subscriber cannot receive data from a Best Effort publisher (QoS incompatibility), which would result in no data flow.¹⁵

4.3 Execution Management and Spin Strategy

The `rclcpp_executor` component in Micro-ROS is responsible for managing callbacks and data transmission. A common pattern is to use `rclcpp_executor_spin_some` with a timeout. However, in a sensor-only node (Publisher), there are no incoming subscriptions to service. The executor is primarily performing housekeeping (handling the XRCE-DDS session heartbeats).

If `rclcpp_executor_spin_some` is called with a significant timeout (e.g., 100ms), the Network Task effectively sleeps for that duration, blocking the transmission of the next packet. The timeout should be set to the absolute minimum (e.g., 0 or 1ms) or replaced with a non-blocking spin logic. This ensures that the Network Task spends the majority of its CPU cycles fetching new sensor data and pushing it to the wire, rather than waiting for incoming traffic that will never arrive.¹⁷

4.4 Memory Management and MTU

Micro-ROS utilizes a static memory model to prevent heap fragmentation, which is a common cause of instability in long-running embedded systems. The middleware pre-allocates buffers based on the configured limits.

- **RMW_URXCE_MAX_HISTORY:** This parameter controls the size of the message queue. For a "History: Keep Last, Depth: 1" policy, this parameter should be set to a low value (e.g., 2 or 4) in the `colcon.meta` configuration file. Excessive history wastes limited ESP32 RAM (SRAM) and offers no benefit for this application.
- **MTU (Maximum Transmission Unit):** For UDP over Wi-Fi, the MTU is typically 1500 bytes. A quaternion message is very small (approx. 4 floats + header $\approx$ 64 bytes). Therefore, fragmentation is not an issue. However, ensuring the Micro-ROS Agent and Client agree on the MTU is vital to prevent packet drops at the framing layer.¹⁹

5. Host-Side Algorithmic Compensation and Prediction

Regardless of the extent of embedded optimization, physical constraints dictate a non-zero latency. The sum of sensor fusion time (~5ms), I2C readout (~3ms), RTOS overhead (~1ms), Wi-Fi transmission (~3-10ms), and host OS network stack processing leads to a baseline latency of 15-30ms. In a wireless environment, packet loss and retransmission (at the MAC layer) can cause jitter spikes of 50ms or more. To present a smooth 50+ Hz signal to the robot controller, the Host computer must employ **Dead Reckoning (Signal Prediction)**.

5.1 Rotational Dead Reckoning Theory

Dead reckoning is the process of estimating the current position of a moving object based on

a previously determined position and estimated velocities. In the context of orientation, this involves extrapolating quaternions.

Given a stream of quaternions $q_0, q_1, \dots, q_k$ received at times $t_0, t_1, \dots, t_k$, we wish to estimate the orientation q_{now} at time t_{now} , where $t_{\text{now}} > t_k$.

The fundamental assumption is that for small time steps (like the 20ms gap in a 50 Hz stream), the angular velocity $\vec{\omega}$ is constant.

1. **Differential Rotation:** First, we compute the relative rotation Δq between the two most recent samples:

$$\Delta q = q_k \otimes q_{k-1}^{-1}$$

(where $\otimes$ denotes quaternion multiplication and q^{-1} is the conjugate q^* for unit quaternions).

2. **Angular Velocity Extraction:** We convert Δq into an axis-angle representation $(\theta, \vec{v})$.

$$\theta = 2 \arccos(w_{\Delta q})$$

$$\vec{v} = \frac{\sin(\theta/2)}{\theta} [\text{imaginary part of } \Delta q]$$

The average angular velocity magnitude over the interval $\Delta t = t_k - t_{k-1}$ is $\omega_{\text{mag}} = \theta / \Delta t$. The angular velocity vector is $\vec{\omega} = \vec{v} \cdot \omega_{\text{mag}}$.

3. **Extrapolation:** We project this velocity forward over the latency period $t_{\text{lat}} = t_{\text{now}} - t_k$.

$$\theta_{\text{pred}} = \omega_{\text{mag}} \cdot t_{\text{lat}}$$

$$q_{\Delta \text{pred}} = [\cos(\theta_{\text{pred}}/2), \vec{v} \sin(\theta_{\text{pred}}/2)]$$

$$q_{\text{now}} \approx q_{\Delta \text{pred}} \otimes q_k$$

21

5.2 Implementation Strategies: SLERP vs. NLERP

While the math above describes analytical extrapolation, practical implementation often utilizes interpolation functions for stability.

- **SLERP (Spherical Linear Interpolation):** This is the geometrically correct method for interpolating between quaternions. It follows the "great circle" path on the 4D hypersphere of unit quaternions. It ensures constant angular velocity during the interpolation.
- **NLERP (Normalized Linear Interpolation):** This method linearly interpolates the quaternion components and then re-normalizes the result. It is computationally cheaper than SLERP but does not preserve constant angular velocity (it accelerates in the middle of the arc).

- **Recommendation:** For the Host side (running on a powerful PC), **SLERP** or the direct angular velocity extrapolation derived above is preferred for accuracy. For prediction, we are essentially performing SLERP with a factor $t > 1.0$.

5.3 Kalman Filtering for Jitter Compensation

Extrapolation amplifies noise. If the input data q_k is noisy, the predicted future state will swing wildly. To mitigate this, an **Extended Kalman Filter (EKF)** should be implemented on the Host.

- **State Vector:** $x = [q, \vec{\omega}]^T$ (Orientation quaternion and Angular Velocity).
 - **Prediction Model:** Updates the state based on the constant velocity assumption.
 - **Correction Step:** Updates the state when a new packet arrives from the ESP32.
- The EKF provides a smoothed estimate of $\vec{\omega}$ that is robust to individual noisy packets, making the extrapolation step much more stable. Furthermore, if a packet is dropped, the EKF can continue to predict the state based on the last known velocity, effectively "filling in the gaps" for the robot controller.²²

6. System Verification and Benchmarking

The final phase of optimization is empirical verification. The "50 Hz" target must be verified not just by the transmission rate, but by the *smoothness* of the data.

6.1 Latency Measurement Methodology

End-to-end latency is difficult to measure without an external reference. A recommended validation rig involves:

1. Mounting the IMU on the robot flange.
2. Commanding the robot to move in a sinusoidal pattern.
3. Logging the robot's internal encoder values (ground truth) and the IMU's reported values.
4. Cross-correlating the two signals. The time shift required to align the signals represents the total system latency.

6.2 RViz2 Visualization Pipeline

RViz2 is the standard visualization tool. However, RViz2 itself introduces rendering latency. When debugging, it is crucial to ensure that the TF (Transform) tree is being broadcast correctly. The Host node should broadcast the extrapolated quaternion as a transform from world to hand_link.

- **Visual Check:** A high-speed camera (e.g., 240fps on a smartphone) can record the operator's hand and the screen simultaneously. Counting the frames between the hand stopping and the screen stopping provides a rough latency estimate.

7. Conclusion

The development of an industrial-grade tele-imitation interface using low-cost embedded hardware like the ESP32 requires a rigorous engineering approach that acknowledges and mitigates the specific limitations of the components. The BNO055's I2C clock stretching is not a defect but a characteristic of on-sensor processing; by configuring the ESP32's driver timeouts and electrical pull-ups correctly, it becomes manageable. The jitter inherent in Wi-Fi transmission is not a showstopper; by employing FreeRTOS dual-core scheduling, UDP Best Effort transport, and host-side Kalman filtering, it can be smoothed out.

This report demonstrates that by shifting from a naive "read-and-send" loop to a sophisticated, multi-stage pipeline—encompassing electrical tuning, RTOS scheduling, middleware QoS optimization, and predictive signal processing—the target of 50 Hz stable control is achievable. The resulting system transforms the ESP32 from a simple IoT distinct device into a robust real-time controller capable of driving industrial automation tasks.

Works cited

1. BNO055 issue - adafruit industries, accessed December 19, 2025, <https://forums.adafruit.com/viewtopic.php?t=216645>
2. BNO055 on Esp32 in task - Adafruit forums, accessed December 19, 2025, <https://forums.adafruit.com/viewtopic.php?t=173144>
3. I2C: Add ability to do clock stretching · Issue #352 · esp-rs/esp-hal - GitHub, accessed December 19, 2025, <https://github.com/esp-rs/esp-hal/issues/352>
4. What are the BNO055 with ESP32 I2C issues? - Adafruit forums, accessed December 19, 2025, <https://forums.adafruit.com/viewtopic.php?t=205215>
5. I2C Addresses and Troublesome Chips | Adafruit Learning System, accessed December 19, 2025, <https://learn.adafruit.com/i2c-addresses/troublesome-chips>
6. How to use ESP32 Dual Core with Arduino IDE - Random Nerd Tutorials, accessed December 19, 2025, <https://randomnerdtutorials.com/esp32-dual-core-arduino-ide/>
7. Using ESP32 Dual-Core with RTOS for High-Frequency ADC · micropython · Discussion #15847 - GitHub, accessed December 19, 2025, <https://github.com/orgs/micropython/discussions/15847>
8. Queue Management - - — ESP-Techpedia latest documentation - Espressif Systems, accessed December 19, 2025, <https://docs.espressif.com/projects/esp-techpedia/en/latest/esp-friends/get-started/basic-concepts/common-freertos-api/queue-management.html>
9. ESP32 FreeRTOS Inter Task Communication Tutorial - ControllersTech, accessed December 19, 2025, <https://controllerstech.com/esp32-freertos-inter-task-communication/>
10. ESP32 Arduino Tutorial: FreeRTOS queues performance test - DFRobot, accessed December 19, 2025, <https://www.dfrobot.com/blog-745.html>
11. Speed Optimization - ESP32 - — ESP-IDF Programming Guide v5.5.1

- documentation, accessed December 19, 2025,
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/performance/speed.html>
12. ESP32 - floating point performance - Reddit, accessed December 19, 2025,
https://www.reddit.com/r/esp32/comments/1la8fob/esp32_floating_point_performance/
 13. Adding debug and optimization options to Arduino IDE v2 - ESP32 Forum, accessed December 19, 2025, <https://esp32.com/viewtopic.php?t=31181>
 14. Code optimization - PlatformIO Community, accessed December 19, 2025,
<https://community.platformio.org/t/code-optimization/12451>
 15. How to use custom QoS in micro-ROS, accessed December 19, 2025,
https://micro.ros.org/docs/tutorials/advanced/create_dds_entities_by_ref/
 16. Publishers and subscribers - micro-ROS, accessed December 19, 2025,
https://micro.ros.org/docs/tutorials/programming_rcl_rclc/pub_sub/
 17. Function rcl_executor_spin_some — rcl: Humble 4.0.2 documentation, accessed December 19, 2025,
https://docs.ros.org/en/humble/p/rcl/generated/function_executor_8h_1abbbff1926668e001c2f6f3accbe012945.html
 18. 6.3.4. Executor and timers — Vulcanexus 1.0.0 documentation, accessed December 19, 2025,
https://docs.vulcanexus.org/en/iron/rst/microros_documentation/user_api/user_api_executor_timers.html
 19. Memory profiling - micro-ROS, accessed December 19, 2025,
https://micro.ros.org/docs/concepts/benchmarking/memo_prof/
 20. Middleware Configuration - micro-ROS, accessed December 19, 2025,
https://micro.ros.org/docs/tutorials/advanced/microxrcedds_rmw_configuration/
 21. Keeping a Good Attitude: A Quaternion-Based Orientation Filter for IMUs and MARGs - MDPI, accessed December 19, 2025,
<https://www.mdpi.com/1424-8220/15/8/19302>
 22. Head Orientation Prediction: Delta Quaternions Versus Quaternions - IEEE Xplore, accessed December 19, 2025,
<https://ieeexplore.ieee.org/iel5/3477/5238715/05033784.pdf>
 23. Multi-Step Prediction of Physiological Tremor With Random Quaternion Neurons for Surgical Robotics Applications - DR-NTU, accessed December 19, 2025,
<https://dr.ntu.edu.sg/server/api/core/bitstreams/7e129906-54a1-4c27-9fe3-a8cb0ab128ec/content>
 24. mbrossar/ai-imu-dr: AI-IMU Dead-Reckoning - GitHub, accessed December 19, 2025, <https://github.com/mbrossar/ai-imu-dr>